

Internship Report on
Crop-wise Abiotic Feature Importance for Soil Disease Prediction

By

M. Nandhitha Sri 2023002415

A.Sai Sahithya 2023002367

K.Tejaswini 2023002495

N.Rishitha 2023003021

G.Deepthi 2023002042

Course code : INTN2333



Department of Computer Science and Systems Engineering
GITAM School of CSE, GITAM (Deemed to be University)
Bengaluru, Karnataka, India

GITAM SCHOOL OF COMPUTER SCIENCE (GSCE)
DEPARTMENT OF COMPUTER SCIENCE & SYSTEMS
ENGINEERING



CERTIFICATE

This is to certify that the internship report entitled “**Crop-wise Abiotic Feature Importance for Soil Disease Prediction**” is a bonafide record of work carried out by M. Nandhitha Sri (2023002415), A. Sai Sahithya (2023002367), K. Tejaswini (2023002495), N. Rishitha (2023003021), and G. Deepthi (2023002042) submitted in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

Name of Guide: Dr. Shobha Rani.N

Designation: Assistant Professor

Department: Artificial Intelligence and Data Science

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible, whose consistent guidance and encouragement crowned our efforts with success.

We consider it our privilege to express our gratitude to all those who guided us in the completion of the project.

We express our gratitude to Director Prof. Y. Vamshidhar, for having provided us with the golden opportunity to undertake this project work in their esteemed organization.

We sincerely thank Dr. Smita Sandeep Darandale, HOD, Department of Computer Science and Systems Engineering, Gitam School of Computer Science, Bengaluru, for the immense support given to us.

We express our gratitude to our project guide Dr.Shobha Rani.M,School of Computer Science and Engineering, Gandhi Institute of Technology and Management, Bengaluru, for their support, guidance, and suggestions throughout the project work.

Student Name	Registration No.
---------------------	-------------------------

M.Nandhitha Sri	2023002415
-----------------	------------

A.Sai Sahithya	2023002367
----------------	------------

K.Tejaswini	2023002495
-------------	------------

N.Rishitha	2023003021
------------	------------

G.Deepthi	2023002042
-----------	------------

TABLE OF CONTENTS

Internship Cover Page	I
Certificate	II
Acknowledgement	III
Table of Contents	IV
1.Abstract	V
2. Introduction	5
3. Literature Survey	8
4. Problem Statement	11
5. Objectives	11
6. System Architecture	12
7. Dataset Description	14
8. Data Pre Processing	16
9. Exploratory Data Analysis	20
10. Machine Learning Model	25
11. Model Training	29
12. Evaluation Metrics.....	34
13. Deployment.....	41
14.Challenges	46
15. Future Work.....	46
16. References.....	47

Abstract

Agriculture is a fundamental sector that supports the livelihood of a large portion of the global population and plays a crucial role in ensuring food security. However, crop productivity is significantly affected by various diseases caused due to unfavorable environmental conditions and nutrient imbalances in the soil. Early identification of such diseases is essential to minimize crop loss and improve yield quality.

This project presents a machine learning-based approach to predict crop diseases using abiotic factors such as soil nutrients and environmental conditions. The dataset used for this study includes parameters like Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall.

A classification model using XGBoost is developed to categorize crops into four classes: *No Disease*, *Fungal*, *Bacterial*, and *Viral*. To address class imbalance, SMOTE (Synthetic Minority Oversampling Technique) is applied. The model is evaluated using performance metrics such as accuracy, precision, recall, and F1-score.

Furthermore, the trained model is deployed using a Flask web application, enabling real-time predictions. A Power BI dashboard is also created to visualize feature importance and disease patterns.

The results demonstrate that the model achieves high accuracy and can be effectively used for early disease prediction, assisting farmers in decision-making.

2. INTRODUCTION

2.1 Importance of Agriculture

Agriculture is one of the oldest and most important sectors, forming the backbone of food production and economic stability, especially in countries like India. It supports a large portion of the population directly or indirectly and contributes significantly to GDP.

With increasing global population, food demand is rising rapidly. However, agricultural productivity is affected by challenges such as climate change, soil degradation, water scarcity, and crop diseases. Hence, adopting modern technologies is essential to improve yield and sustainability.

2.2 Role of Soil Nutrients and Environmental Factors

Crop growth depends on soil nutrients and environmental conditions. Soil provides essential nutrients required for plant development.

Major nutrients include:

- **Nitrogen (N):** Supports leaf growth and photosynthesis. Deficiency causes yellowing and stunted growth.
- **Phosphorus (P):** Important for root development and energy transfer. Deficiency leads to weak roots and slow growth.
- **Potassium (K):** Helps in water regulation and disease resistance. Deficiency increases plant stress.

Environmental factors such as temperature, humidity, rainfall, and soil pH also affect crop health. Imbalance in these factors increases disease risk and reduces productivity.

2.3 Impact of Crop Diseases

Crop diseases significantly reduce agricultural yield and quality. They are caused by fungi, bacteria, and viruses.

Key impacts include:

- Reduced crop yield
- Poor quality produce
- Economic losses to farmers

Types of diseases:

- **Fungal:** Spread in humid conditions
- **Bacterial:** Cause leaf spots, wilting, and rot

- **Viral:** Spread by insects and are difficult to control

Early management is essential to minimize damage.

2.4 Need for Early Detection

Early detection helps prevent large-scale crop loss. Traditional manual methods have limitations such as being time-consuming, error-prone, and not scalable.

Therefore, automated systems are needed to provide timely and accurate disease detection, helping farmers take preventive actions quickly.

2.5 Machine Learning in Agriculture

Machine Learning enables data-driven solutions in agriculture. It helps in predicting diseases, yield estimation, soil analysis, and weather forecasting.

Algorithms like Decision Trees, Random Forest, and XGBoost are widely used for classification tasks. They handle large datasets effectively and provide high accuracy.

In this project, ML is used to predict crop diseases based on soil and environmental conditions.

2.6 Overview of the Proposed System

The system predicts crop diseases using abiotic factors through a complete ML pipeline.

Key components:

- **Data Collection:** Soil nutrients (N, P, K), temperature, humidity, pH, rainfall
- **Data Preprocessing:** Cleaning, encoding, normalization, and SMOTE for balancing
- **Model Development:** XGBoost algorithm for prediction
- **Evaluation:** Accuracy, precision, recall, and F1-score

- **Deployment:** Flask-based web application for real-time prediction
- **Visualization:** Power BI dashboard for insights and trends

The system provides an end-to-end solution from data processing to deployment for practical agricultural use.

3. LITERATURE SURVEY

The literature survey reviews existing techniques for crop disease detection, ranging from traditional methods to advanced machine learning and deep learning approaches. It helps identify gaps and justify the proposed system.

3.1 Traditional Methods

Crop disease detection was initially performed through manual inspection by farmers and experts.

Farmers identify symptoms such as leaf discoloration, spots, wilting, and abnormal growth, and take necessary actions like applying fertilizers or pesticides. In some cases, agricultural experts are consulted for better diagnosis.

Limitations:

- Subjective and depends on experience
- Time-consuming for large fields
- Late detection after visible symptoms
- Not scalable
- Low accuracy in early-stage detection

These limitations highlight the need for automated systems.

3.2 Machine Learning Approaches

Machine learning helps analyze agricultural data and predict diseases using patterns in datasets.

- **Decision Trees:** Simple and interpretable but prone to overfitting
- **Random Forest:** Ensemble method with higher accuracy and better robustness
- **Support Vector Machine (SVM):** Effective for classification and smaller datasets

Advantages:

- Handles large datasets
- High accuracy
- Reduces human intervention
- Scalable

However, ML models require proper preprocessing and feature engineering.

3.3 Deep Learning Approaches

Deep learning uses neural networks, especially for image-based disease detection.

- **CNN (Convolutional Neural Networks):** Automatically extracts image features and achieves high accuracy in leaf disease detection.

Applications:

- Leaf disease detection using images
- Real-time diagnosis systems
- Automated monitoring

Limitations:

- Requires large datasets
- High computational cost
- Difficult to interpret
- Longer training time

3.4 Research Paper Summary

- **Paper 1:** ML-based prediction using Random Forest achieved moderate accuracy but needed better data balancing.
- **Paper 2:** CNN-based image classification achieved high accuracy but required large datasets and high computation.
- **Paper 3:** IoT-based system enabled real-time monitoring but had high cost and hardware dependency.

Method	Accuracy	Cost	Complexity	Scalability	Interpretability
Manual Methods	Low	Low	Low	Low	High
Machine Learning	Medium to high	Medium	Medium	High	Medium
Deep Learning	High	High	High	High	Low

Discussion

- Manual methods are simple and inexpensive but lack accuracy and scalability.
- Machine learning methods provide a balance between accuracy, cost, and complexity, making them suitable for structured datasets.
- Deep learning methods offer the highest accuracy but require significant computational resources and large datasets.

3.6 Research Gap Identification

From the above survey, the following gaps are identified:

- Limited use of abiotic factors for disease prediction
- Lack of balanced datasets in many studies
- High dependency on image data in deep learning approaches
- Need for lightweight and deployable models

3.7 Justification of Proposed Work

To address these gaps, the proposed system:

- Uses abiotic features (soil + environmental data)
- Applies SMOTE to handle imbalance
- Uses XGBoost for better accuracy
- Provides a deployable solution using Flask
- Includes visualization through Power BI

4. PROBLEM STATEMENT

Crop production is strongly influenced by soil nutrients and environmental conditions such as temperature, humidity, rainfall, soil pH, and nutrient levels (N, P, K). These factors directly affect plant health and increase the risk of diseases like fungal, bacterial, and viral infections.

A major challenge in agriculture is the **early detection of crop diseases**. Farmers mostly depend on visual inspection and experience, which is unreliable since symptoms often appear only in later stages. By that time, significant crop damage and yield loss may already occur.

Another issue is the **lack of automated and data-driven systems**, especially in rural areas where expert guidance and advanced tools are limited. Changing environmental conditions further make manual monitoring difficult and inconsistent.

Existing methods are often time-consuming, labor-intensive, and require expert knowledge. Therefore, there is a need for an **efficient, scalable, and automated system** that can predict crop diseases using easily available abiotic factors.

The proposed solution uses **machine learning techniques** to analyze soil and environmental data and predict crop health conditions. This reduces dependency on manual inspection and enables faster and more accurate decision-making.

In summary, the problem is the absence of an accurate, automated system for early crop disease prediction using abiotic data, which is essential for improving agricultural productivity and reducing losses.

5. OBJECTIVES

The main objective is to develop a **machine learning-based system** for predicting crop diseases using soil and environmental parameters to support better agricultural decision-making.

5.1 Machine Learning Model Development

Build an accurate ML model that classifies crop conditions such as **No Disease, Fungal, Bacterial, and Viral** using structured agricultural data.

5.2 Use of Abiotic Features

Use soil and environmental factors such as:

- Nitrogen (N), Phosphorus (P), Potassium (K)
- Temperature

- Humidity
- Soil pH
- Rainfall

These features make the system practical and suitable for real-world use.

5.3 Handling Data Imbalance with SMOTE

Apply SMOTE to balance the dataset by generating synthetic samples for minority classes. This helps:

- Improve model performance
- Reduce class bias
- Enhance prediction accuracy

5.4 Feature Importance Analysis

Analyze the impact of each feature to understand their role in disease prediction, helping improve model interpretability and agricultural insights.

5.5 Real-Time Deployment

Deploy the model using a **Flask web application** that:

- Accepts user inputs
- Provides real-time predictions
- Offers a simple user interface

Power BI is used for visualization of insights.

5.6 Summary

The project focuses on:

- Accurate ML-based prediction
- Use of abiotic data instead of images
- SMOTE for balancing data
- Feature analysis for insights
- Real-time deployment for practical use

6. SYSTEM ARCHITECTURE

The system architecture of the proposed crop disease prediction system defines a structured workflow connecting user input, data processing, machine learning prediction, and output display. It ensures a simple, scalable, and efficient system for accurate disease prediction.

6.1 Workflow of the System

The system follows a step-by-step pipeline:

Step 1: User Input

Users enter abiotic factors through a web form, including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall.

Step 2: Data Preprocessing

Input data is processed in the backend by:

- Converting values to numeric format
- Scaling/normalizing data if required
- Ensuring correct feature alignment
- Handling missing or invalid inputs

Step 3: Model Prediction

The processed data is passed to the trained **XGBoost model**, which predicts crop condition:

- No Disease
- Fungal Disease
- Bacterial Disease
- Viral Disease

Step 4: Output Display

The prediction result is returned and displayed on the web interface in a simple format for easy understanding.

6.2 System Components

6.2.1 Frontend (HTML/CSS)

The frontend collects user inputs and displays results. It provides a simple and user-friendly interface for entering crop parameters and viewing predictions.

6.2.2 Backend (Flask)

Flask acts as the bridge between frontend and ML model. It:

- Receives input data
- Preprocesses inputs

- Loads trained model (joblib/pickle)
- Sends data for prediction
- Returns output to UI

6.2.3 Machine Learning Model (XGBoost)

XGBoost is the core prediction engine. It:

- Learns patterns from training data
- Predicts disease categories based on inputs
- Handles structured and imbalanced data effectively

It is chosen for its **high accuracy, speed, and robustness**.

6.3 System Flow Summary

User Input → Frontend → Flask Backend → Preprocessing → XGBoost Model → Prediction → Output Display

6.4 Advantages

- Simple and user-friendly design
- Real-time prediction capability
- Modular structure (frontend, backend, ML model)
- Scalable for future improvements
- Easy integration with visualization tools

6.5 Conclusion

The system architecture provides an efficient end-to-end framework for crop disease prediction using machine learning. It combines a user-friendly interface, Flask backend, and XGBoost model to deliver accurate and real-time predictions for agricultural decision-making.

7. DATASET DESCRIPTION

The dataset is a key component of this project as it directly affects model performance and accuracy. A structured agricultural dataset is used to predict crop diseases based on abiotic factors such as soil nutrients and environmental conditions.

7.1 Source of the Dataset

The dataset is taken from the **Kaggle Crop Recommendation Dataset**, a widely used open-source dataset for agricultural machine learning research.

It contains structured tabular data with soil and environmental parameters suitable for supervised classification tasks like crop disease prediction using models such as XGBoost.

7.2 Features Explanation

The dataset includes the following abiotic features:

- **Nitrogen (N):** Supports leaf growth and photosynthesis; imbalance increases disease risk
- **Phosphorus (P):** Helps root development and energy transfer; improves plant resistance
- **Potassium (K):** Strengthens immunity and regulates water balance
- **Temperature:** Affects plant metabolism and disease development
- **Humidity:** Promotes fungal growth under high levels
- **pH:** Controls nutrient availability and soil health
- **Rainfall:** Influences water availability; excess causes fungal risk, deficit causes stress

These features collectively determine crop health conditions.

7.3 Target Classes

The dataset supports multi-class classification with the following outputs:

- **No Disease:** Healthy crops with optimal conditions
- **Fungal Disease:** Caused by fungi, common in high humidity
- **Bacterial Disease:** Causes wilting and rotting due to bacteria
- **Viral Disease:** Spreads through insects and causes stunted growth

7.4 Importance of the Dataset

This dataset is suitable because:

- It contains real agricultural parameters
- Includes both soil and environmental factors
- Supports multi-class classification
- Easy to preprocess and use for ML models
- Helps learn relationships between abiotic factors and crop diseases

8. DATA PREPROCESSING

Data preprocessing is a fundamental stage in the machine learning pipeline that transforms raw agricultural data into a clean, consistent, and model-ready format. Since real-world datasets often contain inconsistencies, noise, and imbalance, preprocessing plays a crucial role in improving model performance, stability, and accuracy. In this project, careful preprocessing is performed to ensure reliable crop disease prediction using abiotic factors.

8.1 Data Cleaning

Data cleaning is performed to improve dataset quality and remove inconsistencies that may negatively affect model learning.

The following operations are carried out:

- Removal of duplicate records to prevent redundant learning and bias
- Identification and correction of inconsistent or invalid values
- Verification of realistic agricultural ranges for all features
- Ensuring uniform structure across all records

Although the dataset is relatively structured, cleaning ensures that the model is trained on high-quality and reliable data.

8.2 Handling Missing Values

Missing values can introduce uncertainty and reduce model accuracy. Therefore, proper handling is essential.

The following techniques are used:

- Numerical missing values are replaced using **mean or median imputation**
- Categorical missing values (if any) are filled using **mode imputation**
- Records with excessive missing data are removed when necessary

This step ensures dataset completeness and prevents loss of important information during training.

8.3 Feature Selection

Feature selection helps in identifying the most relevant variables that contribute significantly to crop disease prediction. It reduces dimensionality and improves computational efficiency.

The selected features include:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- Soil pH
- Rainfall

These features are chosen because they directly influence plant growth and disease susceptibility.

The **target variable** represents crop health categories:

- No Disease
- Fungal Disease
- Bacterial Disease
- Viral Disease

Irrelevant or redundant features are excluded to enhance model efficiency and reduce overfitting.

8.4 Encoding Categorical Data

Machine learning algorithms require numerical inputs; therefore, categorical labels must be converted into numerical form.

In this project:

- Label Encoding is applied to convert disease categories into numeric values
- Each class is assigned a unique integer identifier

Example encoding:

- No Disease → 0
- Fungal Disease → 1
- Bacterial Disease → 2
- Viral Disease → 3

This enables the model to process categorical outputs effectively during training and prediction

8.5 Handling Imbalanced Data using SMOTE

Agricultural datasets often suffer from class imbalance, where certain disease categories have fewer samples compared to others. This leads to biased model predictions.

To solve this problem, **SMOTE (Synthetic Minority Oversampling Technique)** is applied.

Working of SMOTE:

- Selects samples from minority classes
- Identifies nearest neighbors
- Generates synthetic data points between them
- Expands minority class representation until balance is achieved

Advantages of SMOTE:

- Eliminates bias toward majority class (No Disease)
- Improves detection of rare disease classes
- Enhances recall and F1-score
- Reduces overfitting caused by simple duplication

Impact in this project:

SMOTE significantly improves classification performance for Fungal, Bacterial, and Viral disease categories, making the model more realistic for agricultural applications where disease cases are naturally uneven.

8.6 Feature Scaling

Feature scaling is applied to ensure that all variables contribute equally to model training, as different features have different value ranges.

Method used: StandardScaler

It transforms data such that:

- Mean = 0
- Standard deviation = 1

Benefits:

- Prevents dominance of large-scale features (e.g., rainfall vs pH)
- Improves convergence and model stability

- Enhances prediction efficiency
- Ensures uniform contribution of all features

Even though XGBoost is not highly sensitive to scaling, it improves preprocessing consistency.

8.7 Train-Test Split

To evaluate model performance effectively, the dataset is divided into training and testing sets.

Split Ratio:

- 80% Training Data
- 20% Testing Data

Purpose:

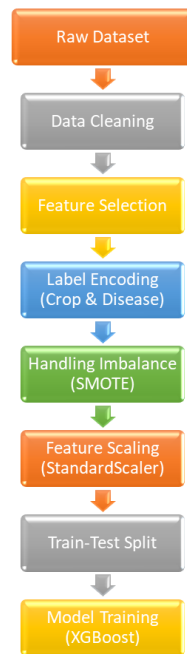
- Training data is used to learn patterns and relationships
- Testing data evaluates performance on unseen data

This ensures that the model generalizes well and avoids overfitting.

8.8 Importance of Preprocessing in This Project

Proper preprocessing ensures:

- High prediction accuracy
- Better generalization on real-world data
- Reduced noise and inconsistencies
- Improved performance of XGBoost model
- Reliable crop disease classification



9. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is a crucial step in the data science pipeline where the dataset is analyzed visually and statistically to understand its structure, patterns, and relationships between variables. It helps in gaining meaningful insights from the data before applying machine learning models.

In this project, EDA is performed to understand how abiotic factors such as soil nutrients and environmental conditions influence crop disease occurrence. The insights derived from EDA also help in validating feature importance and improving model performance.

9.1 Analysis Performed

9.1.1 Correlation Heatmap Analysis

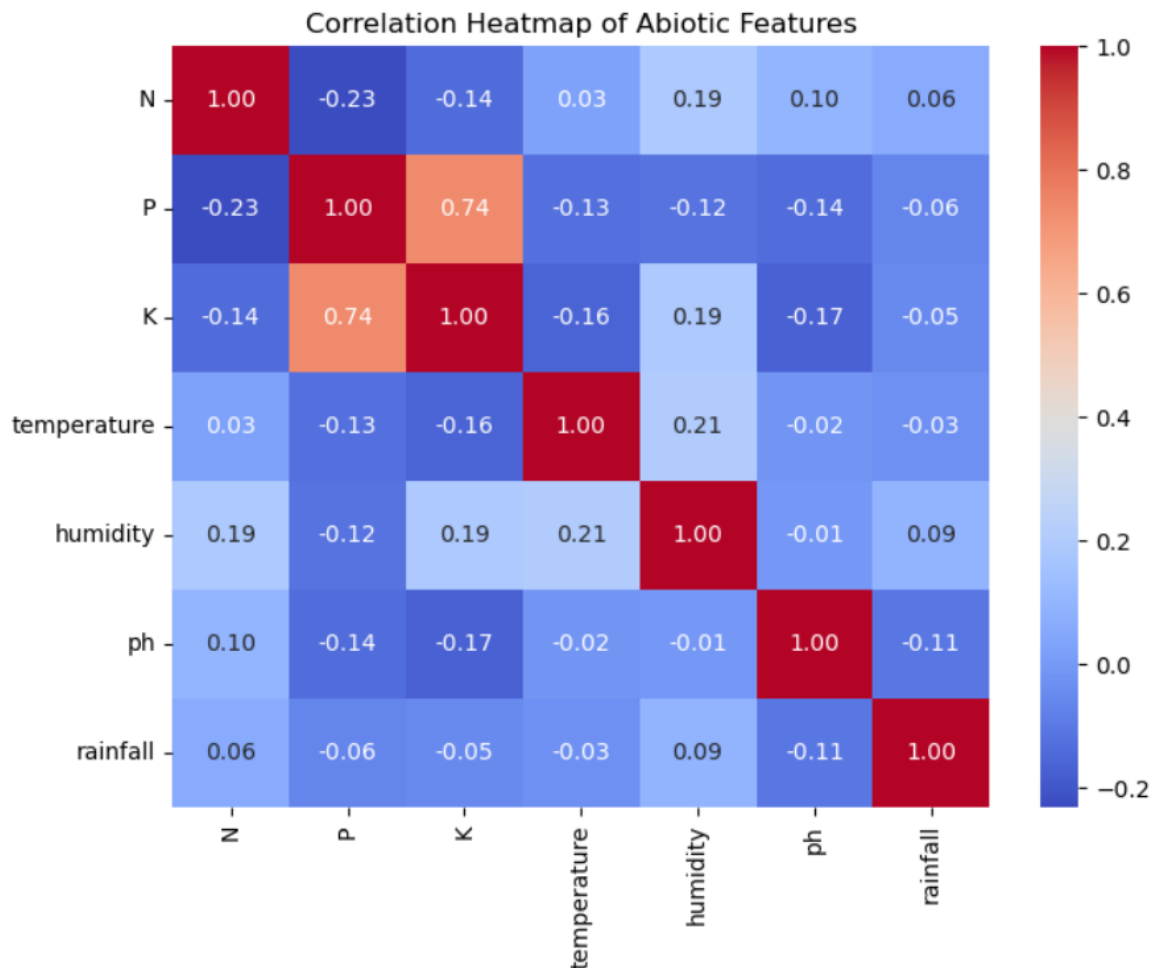
A correlation heatmap is used to visualize the relationship between different numerical features in the dataset. It helps in understanding how strongly variables are related to each other and their influence on crop disease prediction.

In this project, the heatmap is analyzed for features such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.

Key Observations:

- Humidity and rainfall show relatively strong relationships with disease occurrence
- Nutrient levels (N, P, K) show moderate correlation with crop health conditions
- Some features exhibit weak correlation, indicating independent behavior
- Environmental factors play a significant role in disease prediction

The correlation analysis helps identify important features and ensures that the model focuses on the most relevant variables during training.



9.1.2 Distribution Plots

Distribution plots are used to visualize how each feature is distributed across the dataset. These plots help in understanding the range, spread, and frequency of values.

In this project, distribution analysis is performed for:

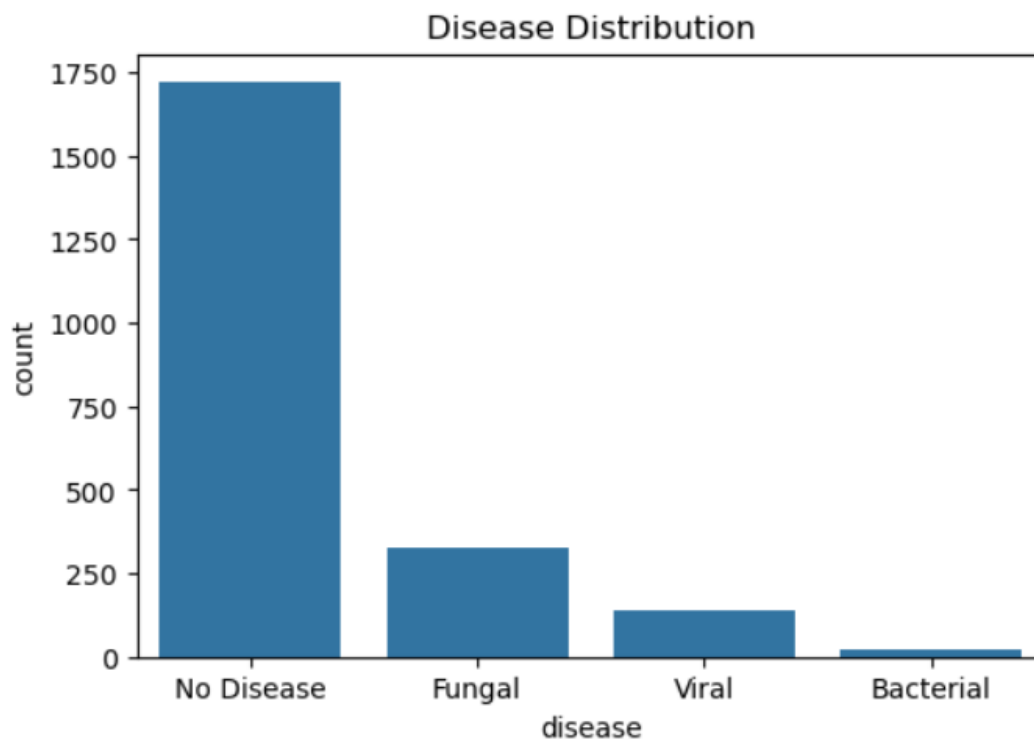
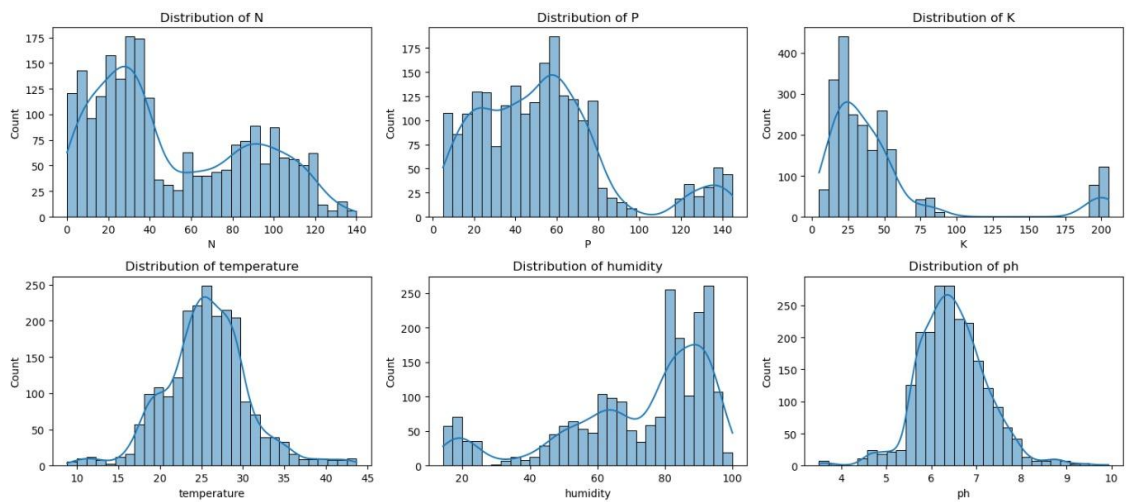
- Nitrogen
- Phosphorus
- Potassium
- Temperature
- Humidity
- pH

- Rainfall

Key Observations:

- Nutrient levels (N, P, K) show variation across different crops
- Temperature values are distributed within a realistic agricultural range
- Humidity shows clustering in higher ranges, indicating its importance in crop growth
- Rainfall distribution varies significantly, showing seasonal influence

These distributions help in identifying whether the dataset is balanced and whether any transformation is required before model training.



9.1.3 Feature Relationship Analysis

Feature relationship analysis is performed to understand how different input variables interact with each other and influence the target variable (crop disease type).

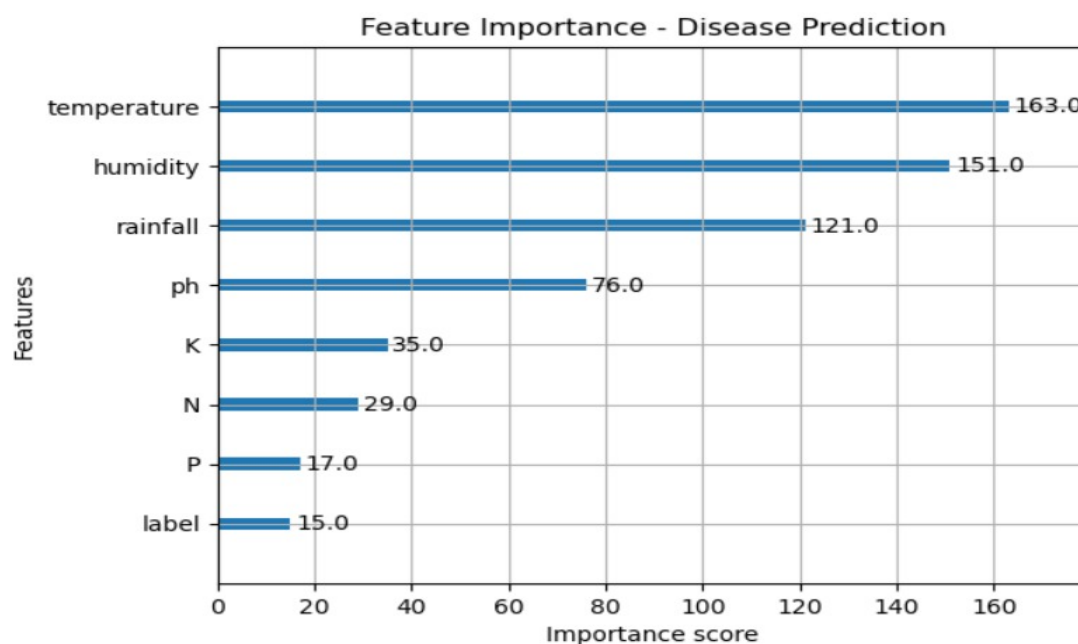
This involves comparing:

- Nutrient levels vs disease occurrence
- Environmental factors vs disease classification
- Combined effect of multiple features

Key Observations:

- High humidity combined with moderate to high temperature increases the likelihood of fungal diseases
- Low potassium levels are often associated with weaker plant immunity and higher disease susceptibility
- Extreme pH levels reduce nutrient absorption, indirectly increasing disease risk
- Rainfall patterns significantly influence bacterial and fungal infections

This analysis helps in understanding the underlying patterns that the machine learning model learns during training.



9.2.4 Environmental Factors Play a Major Role

Environmental conditions significantly influence crop disease development.

- High rainfall increases moisture, promoting fungal and bacterial growth
- Temperature variations affect plant metabolism and resistance
- Combined effects of humidity and rainfall increase disease risk

This confirms that abiotic factors are critical for accurate disease prediction.

9.2.5 Dataset Suitability for Machine Learning

From EDA results, it is observed that:

- The dataset shows clear and meaningful patterns
- Input features are relevant and informative
- Strong relationships exist between features and target classes

Hence, the dataset is well-suited for machine learning models like XGBoost.

9.3 Importance of EDA in This Project

Exploratory Data Analysis (EDA) is important because it:

- Helps understand dataset structure and patterns
- Identifies important features affecting crop health
- Validates relationships between abiotic factors and diseases
- Improves feature selection and model performance
- Provides meaningful agricultural insights

Without EDA, it would be difficult to interpret data and build an accurate prediction model.

10. MACHINE LEARNING MODEL

This section describes the machine learning model used for crop disease prediction. The model selected for this project is XGBoost (Extreme Gradient Boosting), a powerful ensemble learning technique widely used for classification problems due to its high performance, robustness, and efficiency.

10.1 XGBoost

10.1.1 Overview of XGBoost

XGBoost (Extreme Gradient Boosting) is an advanced implementation of gradient boosted decision trees. It is designed to improve speed, performance, and scalability compared to traditional boosting algorithms.

In this project, we used XGBoost as the core classification algorithm to predict whether a crop is healthy or affected by a disease based on soil and environmental features such as:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH level
- Rainfall

XGBoost builds multiple decision trees sequentially, where each new tree corrects the errors made by the previous ones. This process is known as boosting.

10.1.2 Ensemble Technique

XGBoost belongs to the family of **ensemble learning methods**, specifically:

- Bagging (parallel learning, e.g., Random Forest)
- Boosting (sequential learning, e.g., XGBoost, AdaBoost)

In boosting:

1. A weak model (usually a shallow decision tree) is trained.
2. Errors from the previous model are identified.
3. A new model is trained to correct those errors.
4. This process continues iteratively.

Final prediction is obtained by combining all weak learners using a weighted sum.

10.1.3 Gradient Boosting Principle

XGBoost is based on gradient descent optimization. It minimizes a loss function by adding new trees that reduce the residual errors.

The model continuously improves by reducing this loss using gradient-based optimization.

10.1.4 Working Process of XGBoost

The working of XGBoost in this project can be summarized as follows:

Step 1: Data Input

The dataset containing soil and weather parameters is fed into the model.

Step 2: Initial Prediction

A base prediction is made (usually a simple average or initial guess).

Step 3: Compute Residual Errors

Errors between actual and predicted values are calculated.

Step 4: Build Decision Trees

New decision trees are trained to predict these residual errors.

Step 5: Update Model

Predictions are updated by adding outputs from all trees.

Step 6: Final Prediction

The final classification (disease or no disease) is generated.

10.1.5 Why XGBoost is Suitable for This Project

XGBoost is selected because:

- It performs well on structured/tabular data
- It handles non-linear relationships efficiently
- It reduces overfitting using regularization
- It works well even with limited preprocessing
- It provides high prediction accuracy compared to simple models

10.2 Advantages of XGBoost

10.2.1 High Accuracy

One of the most important advantages of XGBoost is its **high predictive accuracy**. It combines multiple weak learners to form a strong model, significantly improving classification performance in complex datasets like crop disease prediction.

10.2.2 Handles Missing Values

XGBoost has an inbuilt mechanism to handle missing values. During training, it learns the best direction to take when a value is missing, making it highly robust for real-world agricultural datasets where missing or inconsistent values may occur.

10.2.3 Fast and Efficient

XGBoost is optimized for performance:

- Supports parallel processing
- Uses cache optimization
- Implements efficient memory usage

This makes it faster compared to traditional gradient boosting methods.

10.2.4 Regularization to Prevent Overfitting

Unlike many models, XGBoost includes both:

- L1 regularization (Lasso)
- L2 regularization (Ridge)

This helps control model complexity and reduces overfitting, ensuring better generalization to unseen data

10.2.5 Feature Importance Analysis

XGBoost provides feature importance scores, helping to identify which environmental factors (such as humidity or soil nutrients) most influence crop disease prediction.

This is useful for:

- Agricultural decision-making
- Identifying critical soil parameters
- Improving interpretability of the model

10.2.6 Scalability

XGBoost is highly scalable and can handle:

- Large datasets
- High-dimensional data
- Complex feature interactions

This makes it suitable for real-world agricultural datasets that grow over time.

10.2.7 Flexibility

XGBoost supports:

- Classification
- Regression
- Ranking problems

In this project, it is specifically used for Multi classification.

10.3 Model Summary in This Project

In the implemented system, the workflow is:

1. Data collection (soil + climate parameters)
2. Preprocessing (cleaning, encoding, scaling)
3. Train-test split
4. XGBoost model training
5. Model evaluation using metrics such as:
 - Accuracy
 - Precision
 - Recall
 - F1-score
6. Prediction of crop disease status

10.3.1 Output Interpretation

The model output is interpreted as:

0: 'No Disease',

1: 'Fungal',

2: 'Bacterial',

3: 'Viral'

- Based on probability thresholds, the system classifies crop health conditions effectively.

11. MODEL TRAINING & TUNING

This section explains how the machine learning model was trained using the dataset and how hyperparameters were optimized to achieve the best possible performance. The training process is a crucial step in building an accurate and reliable predictive system for crop disease detection.

11.1 Model Training

11.1.1 Overview of Training Process

Model training refers to the process of feeding the prepared dataset into the machine learning algorithm so that it can learn patterns and relationships between input features and output labels.

In this project, the dataset consists of agricultural parameters such as:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH level
- Rainfall

The target variable represents whether a crop is:

- Healthy (No Disease)
- Diseased (Disease Present)

The dataset was first divided into:

- **Training set (80%)**
- **Testing set (20%)**

The training set is used to build the model, while the testing set evaluates its performance on unseen data.

The model used for training is XGBoost, which learns patterns through iterative boosting of decision trees.

11.1.2 Training Procedure

The training process follows these steps:

Step 1: Data Preparation

- Handling missing values
- Encoding categorical values (if any)
- Feature scaling (if required)
- Splitting dataset into train and test sets

Step 2: Model Initialization

The XGBoost classifier is initialized with default or pre-defined parameters.

Step 3: Model Fitting

The training dataset is passed to the model using the `.fit()` function.

During this process:

- The model builds multiple decision trees
- Each tree learns from the errors of the previous one
- The model continuously improves prediction accuracy

Step 4: Learning Patterns

The model identifies relationships between environmental conditions and crop health, such as:

- High humidity + low potassium → increased disease probability
- Extreme pH values → crop stress conditions

Step 5: Model Output

After training, the model becomes capable of predicting whether a crop is healthy or diseased based on new input data.

11.1.3 Objective of Training

The main objective of training is to:

- Minimize prediction error
- Improve generalization on unseen data
- Learn complex non-linear relationships in agricultural data

11.2 Hyperparameters

Hyperparameters are external configuration settings that control the behavior of the machine learning model. Unlike model parameters, they are not learned during training but are set before the training process begins.

In this project, tuning hyperparameters is important to improve the performance of XGBoost.

11.2.1 Learning Rate

The learning rate controls how much the model adjusts after each iteration.

- A **high learning rate** makes learning faster but may miss optimal solutions.
- A **low learning rate** improves accuracy but requires more training time.

Mathematically, it controls the contribution of each tree in the final prediction.

$$\hat{y} = \hat{y} + \eta f(x) \quad \hat{y} = \hat{y} + \eta f(x)$$

Where:

- η = learning rate
- $f(x)$ = new tree output

A balanced learning rate ensures stable and accurate convergence.

11.2.2 Max Depth

Max depth defines the maximum depth of each decision tree.

- A **higher depth** allows the model to learn complex patterns but may lead to overfitting.

- A **lower depth** reduces complexity but may underfit the data.

In agricultural datasets, max depth helps the model capture interactions between soil and environmental factors such as:

- Temperature vs humidity
- Soil nutrients vs rainfall

Choosing an optimal depth ensures a balance between accuracy and generalization.

11.2.3 Number of Estimators

The number of estimators represents the total number of decision trees built in the model.

- More estimators → better learning capacity but increased computation time
- Fewer estimators → faster training but lower accuracy

Each estimator contributes to correcting the errors of previous trees, making the final prediction more accurate.

11.3 GridSearchCV

11.3.1 Overview

GridSearchCV is a hyperparameter tuning technique used to find the best combination of parameters for a machine learning model. It performs an exhaustive search over a specified parameter grid.

In this project, GridSearchCV is applied to optimize the performance of XGBoost by testing multiple combinations of hyperparameters.

11.3.2 Working of GridSearchCV

The working process includes:

Step 1: Define Parameter Grid

A set of hyperparameters is defined, such as:

- Learning rate values (e.g., 0.01, 0.1, 0.2)
- Max depth values (e.g., 3, 5, 7)

- Number of estimators (e.g., 100, 200, 300)

Step 2: Cross Validation

The dataset is split into multiple folds (e.g., 5-fold cross-validation).

- The model is trained on some folds
- Tested on remaining folds
- Process is repeated multiple times

Step 3: Model Evaluation

Each combination of hyperparameters is evaluated using performance metrics such as accuracy or F1-score.

Step 4: Best Parameter Selection

The combination that produces the best performance is selected as the final model configuration.

11.3.3 Advantages of GridSearchCV

- Automates hyperparameter tuning
- Improves model accuracy
- Reduces manual effort in parameter selection
- Ensures systematic evaluation of all combinations
- Provides robust performance using cross-validation

11.3.4 Importance in This Project

GridSearchCV plays a key role in improving the crop disease prediction model by:

- Selecting optimal learning rate
- Finding best tree depth
- Balancing bias and variance
- Enhancing generalization on unseen agricultural data

11.4 Final Outcome of Training & Tuning

After training and hyperparameter optimization:

- The model achieves higher accuracy
- Overfitting is reduced

- Predictions become more stable and reliable
- The system performs well on real-world agricultural data

12. EVALUATION METRICS

Evaluation metrics are used to measure the performance of a machine learning model. After training the model, it is essential to evaluate how well it performs on unseen data. These metrics help in understanding the correctness, reliability, and robustness of the model.

In this project, the model is evaluated using standard classification metrics such as Accuracy, Precision, Recall, and F1-score. These metrics provide a detailed understanding of how effectively the model predicts crop diseases.

12.1 Evaluation Metrics

12.1.1 Accuracy

Accuracy is one of the most commonly used evaluation metrics. It measures the proportion of correctly predicted instances out of the total instances.

It is defined as:

- $\text{Accuracy} = (\text{Number of Correct Predictions}) / (\text{Total Predictions})$

Explanation:

Accuracy gives an overall idea of how well the model is performing. If the accuracy is high, it means the model is correctly classifying most of the input samples.

However, accuracy alone may not always be sufficient, especially in imbalanced datasets, where one class dominates the others. In such cases, additional metrics are required to evaluate model performance more effectively

12.1.2 Precision

Precision measures how many of the predicted positive cases are actually correct.

- $\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$

Explanation:

Precision focuses on the correctness of positive predictions. In the context of crop disease prediction, precision indicates how many of the predicted disease cases are actually true disease cases.

High precision means the model makes fewer false alarms (false positives), which is important to avoid unnecessary treatments or interventions in agriculture.

12.1.3 Recall

Recall measures how many actual positive cases are correctly identified by the model.

- $\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$

Explanation:

Recall focuses on the model's ability to detect all actual disease cases. In agriculture, high recall is important because missing a disease case (false negative) can lead to severe crop damage.

A model with high recall ensures that most disease cases are detected early, allowing farmers to take timely action.

12.1.4 F1-Score

F1-score is the harmonic mean of Precision and Recall. It provides a balanced measure when both precision and recall are important.

- $\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Explanation:

F1-score is useful when there is an imbalance between precision and recall. It ensures that both false positives and false negatives are considered while evaluating the model.

In crop disease prediction, F1-score is an important metric because it reflects both correctness and completeness of predictions.

12.2 Confusion Matrix

The confusion matrix is a performance evaluation tool used to visualize the accuracy of a classification model. It provides a detailed breakdown of correct and incorrect predictions for each class.

In this project, the confusion matrix is used to evaluate the model's ability to classify crop diseases into the following categories:

- No Disease
- Fungal Disease
- Bacterial Disease
- Viral Disease

Structure of Confusion Matrix

The confusion matrix consists of:

- **True Positives (TP):** Correctly predicted disease cases
- **True Negatives (TN):** Correctly predicted non-disease cases
- **False Positives (FP):** Incorrectly predicted disease cases
- **False Negatives (FN):** Missed disease cases

Importance in This Project

The confusion matrix is very important because:

- It shows detailed class-wise performance
- Helps identify which disease classes are correctly predicted
- Highlights misclassifications between similar disease types
- Provides insights into model bias

Interpretation of Results

From the confusion matrix, the following observations can be made:

- The model correctly classifies most of the crop disease categories
- Misclassifications are minimal due to effective preprocessing and SMOTE balancing
- The model performs well across both majority and minority classes
- False negatives are reduced, which is important for early disease detection

Why Confusion Matrix is Important in Agriculture

In agricultural applications, confusion matrix plays a critical role because:

- Missing a disease (false negative) can lead to crop loss
- Incorrectly predicting disease (false positive) may lead to unnecessary treatment costs
- Balanced performance across all classes is required for practical use

Thus, the confusion matrix provides a complete picture of model effectiveness beyond simple accuracy.

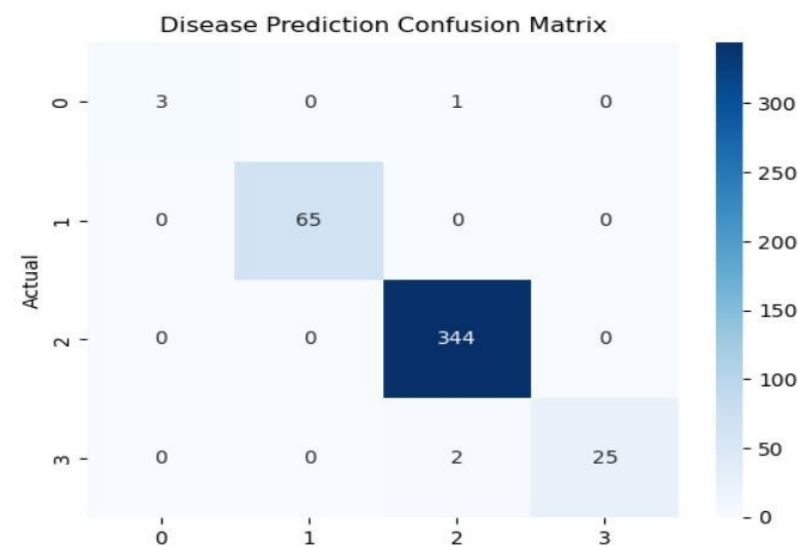
12.3 Summary

In summary, evaluation metrics are essential to measure the performance of the machine learning model. While accuracy provides an overall measure, precision, recall, and F1-score give deeper insights into classification quality. The confusion matrix further helps in understanding class-wise performance and identifying errors.

Together, these metrics confirm that the model is reliable, balanced, and suitable for real-world crop disease prediction tasks.

The XGBoost Classification report and Cofusion matrix

	precision	recall	f1-score	support
0	1.00	0.75	0.86	4
1	1.00	1.00	1.00	65
2	0.99	1.00	1.00	344
3	1.00	0.93	0.96	27
accuracy			0.99	440
macro avg	1.00	0.92	0.95	440
weighted avg	0.99	0.99	0.99	440



The Hyper tuning Classification report and Confusion matrix

Accuracy: 0.9932

```
]]: from sklearn.metrics import classification_report

print("Classification Report:\n")
print(classification_report(
    y_test,
    y_pred,
    target_names=['No Disease', 'Fungal', 'Bacterial', 'Viral']
))
```

Classification Report:

	precision	recall	f1-score	support
No Disease	0.75	0.75	0.75	4
Fungal	1.00	1.00	1.00	65
Bacterial	0.99	1.00	1.00	344
Viral	1.00	0.96	0.98	27
accuracy			0.99	440
macro avg	0.94	0.93	0.93	440
weighted avg	0.99	0.99	0.99	440

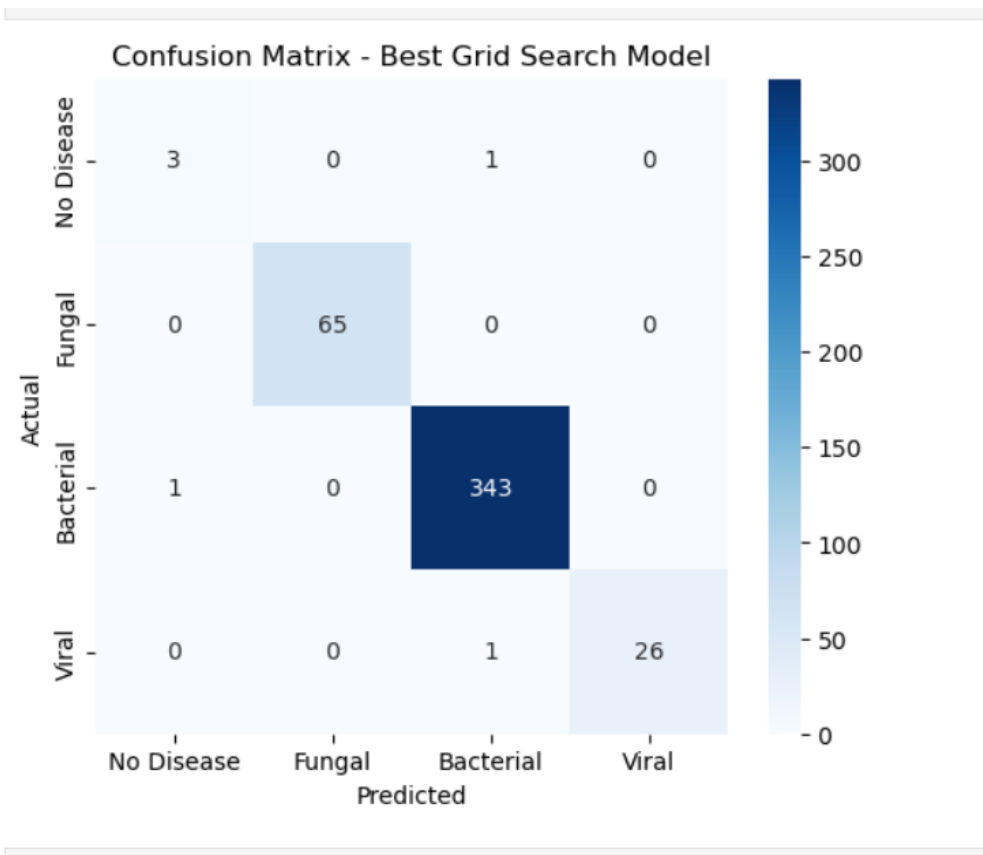


Table Metrics

Class	Precision	Recall	F1-score	Support
0 No Disease	0.750000	0.750000	0.750000	4
1 Fungal	1.000000	1.000000	1.000000	65
2 Bacterial	0.994203	0.997093	0.995646	344
3 Viral	1.000000	0.962963	0.981132	27

POWER BI DASHBOARD

Power BI Dashboard

Crop-wise Abiotic Feature Importance for Soil Disease Prediction



“This is the Power BI dashboard we created for Crop-wise Abiotic Feature Importance for Soil Disease Prediction.”

- “The dashboard provides a visual representation of how different factors affect crop diseases.”
- Explain Components
- “At the top, we can see summary cards showing total number of crops and number of disease types.”
- “On the left, there is a pie chart showing the distribution of diseases like bacterial, fungal, viral, and no disease.”
- “The bar charts show how features like humidity and nitrogen vary across different diseases and crops.”
- “The table below provides detailed crop-wise disease information.”
- Interactivity

- “We also added filters, so users can select a specific disease or crop and analyze the data interactively.”
- Purpose
- “This dashboard helps in understanding which environmental factors influence crop diseases.”
- “It makes the data easy to analyze and supports better decision-making.”

13. DEPLOYMENT

Deployment is the final stage of the machine learning project lifecycle. It involves integrating the trained model into a real-world application so that users can interact with it and obtain predictions. In this project, the crop disease prediction model is deployed using a web-based application built with Flask and a simple HTML frontend.

The deployment ensures that the machine learning model is not limited to theoretical use but is made accessible for practical agricultural applications.

13.1 Flask – Backend Framework

Flask is a lightweight and flexible Python web framework used to develop the backend of this project. It plays a crucial role in handling user requests, processing data, and returning predictions from the trained machine learning model.

Role of Flask in This Project

Flask performs the following functions:

- Receives input data from the user through the frontend form
- Converts the input data into a format suitable for the model
- Loads the trained machine learning model
- Passes the processed input to the model for prediction
- Returns the predicted disease class back to the frontend

Working of Flask Backend

The Flask application follows a simple request-response cycle:

1. User submits input values (N, P, K, temperature, humidity, pH, rainfall)
2. Flask captures the input using HTTP POST request
3. Data is preprocessed in the backend
4. The trained model is loaded and used for prediction
5. Prediction result is sent back to the web page

Advantages of Flask

- Lightweight and easy to implement
- Fast development of web applications
- Easy integration with machine learning models
- Suitable for small and medium-scale deployments
- Supports REST API development

Flask acts as the bridge between the machine learning model and the user interface.

13.2 Frontend – User Interface Design

The frontend of the application is designed using HTML and CSS, which provides a simple and user-friendly interface for input collection.

Purpose of Frontend

The frontend is responsible for:

- Collecting user inputs related to crop conditions
- Providing an interactive and clean interface
- Displaying prediction results clearly
- Ensuring ease of use for farmers or end users

Input Form Design

The frontend consists of an HTML form where users can enter values for:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH level
- Rainfall

Each input field is designed with proper labels to ensure clarity and reduce user confusion.

Frontend Features

- Simple and responsive design
- Structured input form layout
- Clear labels for each feature
- Submit button to trigger prediction
- Display area for showing predicted disease

User Experience Consideration

The interface is designed to be minimal and intuitive so that even non-technical users such as farmers can easily input data and understand the results. The simplicity of the interface enhances usability and accessibility.

13.3 Model Integration Using Joblib

Model integration is a critical step in deployment where the trained machine learning model is connected to the web application.

In this project, Joblib is used to save and load the trained model efficiently.

What is Joblib?

Joblib is a Python library used for:

- Saving machine learning models to disk
- Loading saved models during deployment
- Efficient handling of large numerical data

- Faster serialization compared to pickle

Model Saving Process

After training the machine learning model (XGBoost), it is saved using Joblib:

- The trained model is stored as a file (e.g., `.pkl` or `.joblib`)
- This file contains all learned parameters of the model
- It allows reuse of the model without retraining

Model Loading in Flask

During deployment, the saved model is loaded inside the Flask application:

- The model file is loaded at server startup
- When user inputs data, the model is already available in memory
- Prediction is performed instantly without delay

Prediction Workflow

The integration process works as follows:

1. User submits input through HTML form
2. Flask receives and preprocesses input
3. Joblib loads the trained model
4. Model predicts the disease class
5. Result is returned to frontend
6. Output is displayed to user

Advantages of Using Joblib

- Fast loading of large models
- Memory efficient
- Easy integration with Flask
- Reliable storage of trained models

13.4 Overall Deployment Architecture

The complete deployment system consists of three main components:

1. Frontend Layer

- HTML/CSS interface
- Collects user input
- Displays prediction results

2. Backend Layer

- Flask application
- Handles request processing
- Connects frontend and model

3. Model Layer

- Trained XGBoost model
- Saved using Joblib
- Performs prediction

Workflow Diagram

User Input → HTML Form → Flask Backend → Joblib Model Loading → Prediction → Result Display

13.5 Importance of Deployment

Deployment is important because:

- It converts a machine learning model into a usable application
- Allows real-time prediction of crop diseases
- Makes the system accessible to end users
- Bridges the gap between research and practical application

13.6 Summary

In summary, the deployment phase integrates the trained machine learning model into a web application using Flask as the backend framework and HTML/CSS as the frontend interface. Joblib is used to efficiently load the trained model during runtime. This combination enables real-time, user-friendly crop disease prediction, making the system practical for agricultural use.

14. Challenges

- Imbalance
- Deployment
- Data issues

15. Future Work

- Real-time data
- Image-based detection
- Mobile app

Website Link:

<https://crop-wise-disease-prediction-using.onrender.com/>

The image shows a web application titled "Crop Disease Predictor" with a green plant icon on either side. Below the title is a subtitle: "Select your crop and environmental conditions to predict potential disease." The form contains several input fields: a dropdown menu for "Crop" with "Apple" selected, and text input fields for "N (Nitrogen)", "P (Phosphorus)", "K (Potassium)", "Temperature (°C)", "Humidity (%)", "Soil pH", and "Rainfall (mm)". Each text input field has a placeholder "Enter [parameter] value". At the bottom of the form is a large green button labeled "Predict Disease".

16.References:

- Crop Recommendation Dataset (Kaggle)
<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>
- XGBoost Documentation
<https://xgboost.readthedocs.io/>
- scikit-learn Documentation
<https://scikit-learn.org/stable/>
- Flask Documentation
<https://flask.palletsprojects.com/>
- Python Libraries Documentation
- pandas: <https://pandas.pydata.org/>
- numpy: <https://numpy.org/>
- joblib: <https://joblib.readthedocs.io/>

- Research Papers on Crop Disease Prediction
<https://ieeexplore.ieee.org/>